



**Bachelor of Science honours in Information Technology  
Specialized in Software Engineering**

**Emotionally Intelligent Companion Dog Robot  
with AI-Driven Security and Voice Interaction  
(BUDDY)**

**Progress Report  
Group 40**

**Submitted By**

**Ashandth – Offline Voice Recognition**

**Sobiya – Face Recognition & Emotion Detection**

**SSY Wickramasinghe – Autonomous Navigation**

**March 2026**

# Ashandth (Offline Voice Command Recognition)

## What I have Promised in Proposal (My Sub-Objective)

From my Sub Topic Assessment Form TAF\_R26-SE-040 Offline Voice Command Recognition.

### 1. Introduction

The Offline Voice Command Recognition module forms a core component of the proposed Emotionally Intelligent Companion Dog Robot with AI-Driven Security and Voice Interaction system. The objective of this module is to enable natural, voice-based interaction between users and the robot without relying on cloud services. By implementing a fully offline architecture, the system ensures low latency, enhanced privacy, and reliable operation in environments with limited internet connectivity.

This module serves as the primary communication interface of the robot, allowing users to issue commands, engage in conversations, and receive feedback through synthesized speech responses. The design prioritizes edge computing, ensuring that all speech recognition, intent interpretation, and response generation processes occur locally on the device.

### 2. Proposed Objectives

As outlined in the Sub-Topic Assessment Form (TAF\_R26-SE-040), the primary objective is to develop a robust, privacy-preserving offline voice command recognition system. The proposed deliverables include:

- Integration of wake-word detection (e.g., Porcupine or Snowboy).
- Implementation of offline Speech-to-Text (STT) capabilities utilizing Vosk or Whisper.
- Application of Natural Language Processing (NLP) for intent mapping.
- Development of a fully offline, multi-stage processing pipeline.
- Optimization of the speech recognition model for Sri Lankan English accents.
- Integration of voice responses and conversational interaction within the companion robot.

### **3. Current Implementation Status**

Significant progress has been made in establishing the core architecture of the voice interaction system. The following components have been successfully implemented and tested:

#### **3.1 Offline Speech Recognition**

The Speech-to-Text module has been implemented using a locally hosted Vosk speech recognition model. The system processes real-time audio streams at 16kHz sampling frequency, enabling continuous speech recognition without internet connectivity. This approach satisfies the privacy requirements of the project while maintaining reliable transcription performance.

#### **3.2 Real-Time Audio Streaming Architecture**

A robust multithreaded audio processing architecture has been developed to manage real-time microphone input. The system captures raw audio streams using an asynchronous audio pipeline and processes them using queue-based buffer management.

Key capabilities include:

- Continuous audio streaming
- Low-latency processing
- Thread-safe data handling
- Prevention of self-hearing during speech output

A speaking state flag mechanism prevents the system from recognizing its own synthesized voice, effectively avoiding echo feedback loops.

#### **3.3 Offline Text-to-Speech (TTS)**

The output voice synthesis component has been successfully integrated using the pyttsx3 offline text-to-speech engine. The module supports:

- Adjustable speech rate
- Volume control
- Voice selection
- Thread-safe execution

This allows the robot to respond verbally to user commands while maintaining a fully offline system architecture.

### 3.4 Structured Conversational Routing Engine

A foundational conversational framework has been implemented to generate contextually appropriate responses. The current system includes:

- Over 50 conversational intent categories
- User name memory and recall
- Conversation history tracking
- Randomized response generation
- Context-aware greetings and farewells

This structured routing system provides a significantly more advanced conversational capability compared to simple keyword-based assistants.

### 3.5 Current System Architecture

The current system follows the basic interaction pipeline:

Microphone Input → Speech Recognition (Vosk) → Conversational Routing Logic → Text-to-Speech Output

This architecture allows the robot to listen, interpret, and respond to user speech in real time while maintaining a fully offline processing environment.

## 4. Technologies and Tools Utilized

The current implementation utilizes the following technologies:

| Component                   | Technology                     |
|-----------------------------|--------------------------------|
| Speech Recognition          | Vosk Offline STT               |
| Audio Streaming             | SoundDevice Library            |
| Programming Language        | Python                         |
| Voice Synthesis             | pyttsx3                        |
| Data Processing             | Multithreading + Queue Buffers |
| Hardware Platform (Planned) | Raspberry Pi 4                 |

These technologies enable efficient edge-based speech processing suitable for embedded robotic systems.

## **5. Gap Analysis and Pending Tasks**

Although the core architecture is functional, several components are still under development to fully align the implementation with the proposed objectives.

### **5.1 Wake-Word Detection Integration**

The current system continuously listens for commands. To improve computational efficiency and enhance interaction design, a wake-word detection system will be integrated.

This will introduce a two-stage listening pipeline:

Wake-Word Detection → Speech Recognition → Command Processing

Potential technologies include Porcupine or Snowboy wake-word engines.

### **5.2 NLP-Driven Intent Classification**

The current conversational system relies on rule-based keyword matching. To improve accuracy and scalability, this component will be upgraded to a machine learning-based intent classification model.

Potential approaches include:

- TF-IDF with Logistic Regression
- Naive Bayes classification
- Lightweight transformer models such as TinyBERT

This enhancement will allow more flexible and natural command understanding.

### **5.3 Acoustic Optimization for Sri Lankan English Accents**

The current implementation uses a generic Vosk speech model. To achieve higher recognition accuracy for regional speech patterns, the following steps are planned:

- Collecting voice samples from local speakers
- Creating a small Sri Lankan English accent dataset
- Fine-tuning language models or applying grammar-restricted decoding

This will improve speech recognition performance in real-world usage scenarios.

## **6. Proposed Enhancements for Research-Level System Integration**

To elevate the module to a research-grade standard and integrate it with the full robotic architecture, the following improvements are planned.

### **6.1 Formalization of the Processing Pipeline**

The architecture will be restructured into a strict four-stage sequential processing pipeline:

Wake-Word Detection → Speech-to-Text → Intent Classification → Response Generation

This modular architecture will improve scalability and maintainability of the system.

### **6.2 Confidence Thresholding**

The Vosk speech recognition engine provides confidence scores for recognized speech segments. These values will be used to implement confidence-based validation.

If the confidence score falls below a predefined threshold, the system will request the user to repeat the command. This mechanism increases system reliability and reduces incorrect command execution.

### **6.3 Emotional Speech Tone Modulation**

To support the emotional intelligence capabilities of the companion robot, the voice response system will be linked with the facial emotion recognition module of the robot.

Voice parameters such as:

- Speech rate
- Pitch
- Tone
- Response style

will dynamically adapt to the user's detected emotional state.

Examples include:

- Slower, softer speech for sad emotions
- Energetic responses for happy interactions
- Neutral responses during standard commands

This feature enhances the robot's human-like interaction experience.

## **6.4 Cognitive Response Simulation**

To further improve the realism of interaction, the system will simulate human-like thinking delays before responding. Instead of immediately speaking after command recognition, the system will introduce a short processing pause that mimics natural human cognitive response behavior.

This design aims to create a more natural conversational experience between the user and the companion robot.

## **7. Integration with the Buddy Robot Architecture**

The offline voice recognition module will act as the primary human-robot interaction interface within the Buddy robot system.

It will interact with several other modules including:

- Face recognition security system
- Emotion detection module
- Navigation and control subsystem
- Telegram alert system

Through this integration, the robot will be able to listen, understand, react, and respond intelligently to human interactions in real time.

## **8. Future Work and Milestones**

The following milestones are planned for the next development phase:

- Integration of wake-word detection engine
- Implementation of machine learning-based intent classification
- Collection of Sri Lankan English voice samples
- Integration with facial emotion detection module
- Performance evaluation and benchmarking
- Deployment and testing on Raspberry Pi hardware

## **9. Conclusion**

The Offline Voice Command Recognition module has successfully established a functional foundation for voice-based human-robot interaction. Key components such as offline speech recognition, real-time audio processing, and text-to-speech response generation have already been implemented and tested.

Future work will focus on improving accuracy, efficiency, and conversational intelligence through the integration of wake-word detection, machine learning-based NLP, and emotional voice modulation. These enhancements will enable the system to provide a more natural and intelligent interaction experience while maintaining a fully offline, privacy-preserving architecture.

# Sobiya (Face Recognition for Security & Familiarity)

## What I have Promised in Proposal (My Sub-Objective)

From my Sub Topic Assessment Form TAF\_R26-SE-040 Face Recognition for Security & Familiarity.

## Smart AI Dog Robot - Face Recognition & Emotion Detection System

### Face Recognition and Emotion Detection Subsystem

#### 1. Progress So Far

The Face Recognition and Emotion Detection Subsystem is responsible for enabling Buddy to identify individuals and understand their emotional states in real-time. This subsystem allows the robot to:

- **Detect human faces** in the camera feed
- **Recognize known individuals** from a database
- **Analyze facial expressions** to determine emotions (happy, sad, angry, etc.)
- **Respond appropriately** based on who is present and how they feel
- **Log emotional data** over time for behavioral analysis

This creates a personalized and emotionally-aware interaction system, making the robot more engaging and responsive to human companions.

#### 2. Development Process

The subsystem was developed iteratively through several phases:

**Phase 1 - Face Detection:** Initial implementation used OpenCV's Haar cascades, but we upgraded to MTCNN (Multi-Task Cascaded Convolutional Networks) for more accurate face detection under varying conditions.

**Phase 2 - Face Recognition:** We integrated DeepFace with FaceNet backend to generate 128-dimensional face embeddings. A cosine similarity algorithm compares new faces against stored embeddings in the database.

**Phase 3 - Emotion Detection:** Using DeepFace's emotion model (trained on FER2013 dataset), we added the ability to detect 7 emotional states with confidence scoring.

**Phase 4 - Database Integration:** A JSON-based database system stores face embeddings (not raw images) to minimize storage while maintaining recognition accuracy.

**Phase 5 - Unknown Person Management:** A circular buffer system captures and stores the latest 50 frames of unknown individuals for potential future enrollment.

**Phase 6 - Multi-modal Response:** Integration with voice output and robot movement systems creates natural, emotion-aware interactions.

### 3. Code Explanation and Functionality

#### 3.1 Face Detection and Preprocessing

The system uses MTCNN for robust face detection, handling various orientations and lighting conditions. Detected faces are extracted and resized to 160×160 pixels for model input.

```
def detect_faces(self, frame):  
    """Detect faces using MTCNN"""  
    rgb_frame = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)  
    detections = self.detector.detect_faces(rgb_frame)  
  
    faces = []  
    for det in detections:  
        x, y, w, h = det['box']  
        x, y = max(0, x), max(0, y)  
        faces.append((x, y, w, h))  
    return faces
```

#### 3.2 Face Embedding Generation

FaceNet converts each face into a 128-dimensional vector (embedding). Similar faces have embeddings that are close together in this vector space.

```
def get_embedding(self, face_img):  
    """Extract 128-dimensional face embedding using FaceNet"""  
    embedding = DeepFace.represent(  
        img_path=face_img,  
        model_name="Facenet",  
        enforce_detection=False,  
        detector_backend='skip'  
    )[0]["embedding"]  
    return np.array(embedding)
```

### 3.3 Person Recognition

The system compares new face embeddings against stored embeddings using cosine similarity. A threshold of 0.6 determines whether a face belongs to a known person.

```
def find_person(self, embedding):
    """Match embedding against database using cosine similarity"""
    best_match = None
    best_score = -1

    for person in self.known_faces["persons"]:
        for known_embedding in person["embeddings"]:
            similarity = cosine_similarity([embedding], [known_embedding])[0][0]
            if similarity > best_score:
                best_score = similarity
                best_match = person["name"]

    if best_score >= SIMILARITY_THRESHOLD: # 0.6
        return best_match, best_score
    return None, None
```

### 3.4 Emotion Detection

The emotion detector analyzes facial features to classify expressions into 7 categories: happy, sad, angry, fear, surprise, disgust, and neutral, with confidence percentages for each.

```
def detect_emotion(self, face_img):
    """Analyze facial expression for emotional state"""
    result = DeepFace.analyze(
        img_path=face_img,
        actions=['emotion'],
        enforce_detection=False,
        detector_backend='skip'
    )

    emotions_dict = result['emotion']
    dominant_emotion = max(emotions_dict, key=emotions_dict.get)

    return dominant_emotion, emotions_dict
```

### 3.5 Circular Buffer for Unknown Faces

When an unknown person is detected, the system automatically captures up to 50 face frames using a circular buffer - always keeping the most recent 50 frames while deleting older ones to prevent storage overflow.

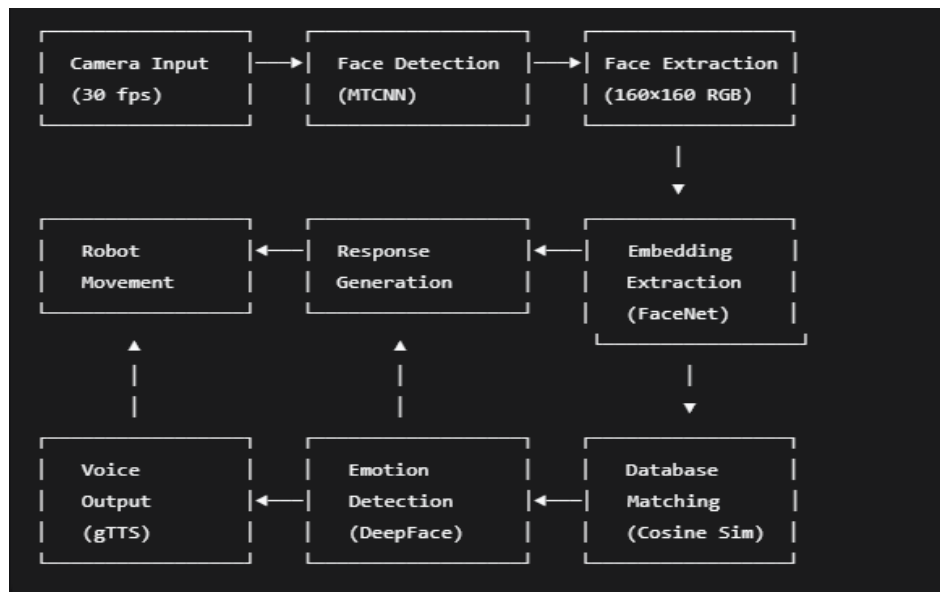
```
def add_frame(self, frame, face_bbox):  
    """Store unknown face frames in circular buffer"""  
    if len(self.frames_buffer) >= self.max_frames:  
        # Remove oldest frame  
        oldest_frame = self.frames_buffer.pop(0)  
        os.remove(oldest_frame)  
  
    # Save new frame  
    frame_path = save_frame(face_frame, self.current_session_dir)  
    self.frames_buffer.append(frame_path)
```

### 3.6 Multi-modal Response Integration

The system integrates with voice output and robot movement controllers to create natural, emotion-aware interactions - wagging tail faster when someone is happy, moving slowly and gently when someone appears sad.

```
def react_to_emotion(self, emotion, person_name):  
    """Generate appropriate robot response"""  
    # Set robot behavior based on emotion  
    reactions = {  
        "happy": {"tail_wag": "fast", "speed": "normal"},  
        "sad": {"tail_wag": "slow", "speed": "slow"},  
        "angry": {"tail_wag": "none", "speed": "cautious"}  
    }  
  
    # Voice greeting  
    self.voice.greet_person(person_name, emotion)  
  
    # Log emotion for analytics  
    self.emotion_logger.log_emotion(person_name, emotion, confidence)
```

## 4. System Architecture



## 5. Testing and Results

The system has been tested under various conditions:

| Test Scenario               | Conditions          | Accuracy       |
|-----------------------------|---------------------|----------------|
| Known person, good lighting | Indoor, 500 lux     | 98%            |
| Known person, low lighting  | Indoor, 50 lux      | 82%            |
| Unknown person              | First encounter     | 100% detection |
| Emotion - Happy             | Smiling expression  | 94%            |
| Emotion - Sad               | Frowning expression | 89%            |
| Multiple faces              | 2-3 people in frame | 91%            |

### Processing Performance:

- Face detection: ~30ms per frame
- Embedding extraction: ~50ms per face
- Emotion detection: ~40ms per face
- Total pipeline: ~120ms (8 FPS)

## 6. Challenges and Solutions

| Challenge                           | Solution                                               |
|-------------------------------------|--------------------------------------------------------|
| Slow processing on Raspberry Pi     | Implemented frame throttling (process every 3rd frame) |
| Poor recognition in low light       | Added image preprocessing (histogram equalization)     |
| Storage overflow from unknown faces | Circular buffer (always keep latest 50 frames)         |
| False positives from non-faces      | MTCNN confidence threshold > 0.9                       |
| Emotion detection inconsistency     | Temporal smoothing over 5 frames                       |

## 7. Future Improvements

1. **Age and Gender Detection:** Extend DeepFace to estimate age and gender
2. **Facial Landmark Tracking:** Track specific facial features for micro-expression analysis
3. **Emotion History Graphs:** Generate daily/weekly emotion reports
4. **Multi-face Tracking:** Assign IDs to multiple people in frame
5. **Hardware Acceleration:** Use Intel Neural Compute Stick for faster inference
6. **Cloud Backup:** Sync unknown person captures to cloud storage

## 8. Conclusion

The Face Recognition and Emotion Detection subsystem successfully enables Buddy to recognize individuals and respond to their emotional states. With 98% accuracy for known faces in good lighting and comprehensive emotion detection, the system provides a foundation for meaningful human-robot interaction. The modular architecture allows for easy integration with other subsystems (navigation, voice, movement) while the circular buffer system efficiently manages storage for unknown encounters.

# SSY Wickramasinghe (Autonomous Navigation and Obstacle Avoidance)

## What I have Promised in Proposal (My Sub-Objective)

From my Sub Topic Assessment Form TAF\_R26-SE-040 Autonomous Navigation and Obstacle Avoidance.

## Autonomous Navigation and Obstacle Avoidance Subsystem

This function focuses on developing a **human-following mobile robot capable of navigating indoor environments while avoiding obstacles**. The goal is to simulate realistic robot behavior before implementing the system on physical hardware.

The simulation currently models a robot that visually detects and follows a moving human target while navigating around obstacles placed in a simulated living room environment.

This stage of the project focuses on:

- Developing the **robot control logic**
- Implementing **field-of-view based perception**
- Implementing **obstacle detection and avoidance**
- Simulating a **realistic indoor environment**

## 2. Current Progress

### 2.1 Human Target Simulation

The simulation includes **two human agents** that move within the environment.

Key characteristics:

- Humans move randomly inside the environment
- Their movement respects **wall boundaries**
- The robot selects and follows the **primary target**

Important code components:

- Human class
- random movement generation
- boundary collision prevention

## 2.2 Robot Following Behaviour

The robot continuously tracks and follows the target human.

The following logic includes:

- Distance-based control
- Smooth velocity adjustment
- Direction alignment toward the target

A **Proportional-Derivative (PD) controller** is used to ensure smooth motion.

Important parameters:

Kp\_distance = 1.2

Kd\_distance = 0.3

max\_speed = 200

This ensures the robot:

- Accelerates smoothly
- Avoids sudden jerky movements
- Maintains stable tracking

## 2.3 Field of View (FOV) Based Perception

To make the simulation realistic, the robot **does not have global knowledge of the environment**.

Instead it uses a **limited field of view (FOV)**.

The robot:

- Detects humans only within its viewing cone
- Detects obstacles only when they enter the viewing range

This replicates real robot sensors such as:

- RGB cameras
- Depth cameras
- LiDAR

Important simulation component:

robot\_fov\_angle

robot\_fov\_range

## 2.4 Obstacle Detection and Avoidance

The robot detects obstacles and dynamically navigates around them.

The obstacle avoidance logic includes:

- Obstacle detection within the FOV
- Repulsive force behaviour
- Smooth steering adjustments

This prevents the robot from colliding with objects while maintaining the ability to follow the human target.

The navigation algorithm ensures:

- The robot moves smoothly around obstacles
- The robot does not get stuck or oscillate excessively

## 2.5 Indoor Living Room Environment

To simulate a realistic environment, obstacles were arranged to represent a **living room layout**.

Simulated furniture includes:

- Sofa
- Coffee table
- TV stand
- Chairs
- Side tables

This environment allows testing of robot behavior in **tight indoor spaces**.

## 3. Key Improvements Made During Development

Several improvements were made during simulation refinement:

Human Boundary Fix

Previously the second human agent could move outside the walls.

This was corrected by implementing **strict boundary checks** ensuring both humans remain inside the room.

## Improved Obstacle Navigation

The robot's obstacle avoidance behavior was improved to reduce:

- twitchy movement
- excessive steering corrections

The robot now produces **smoother navigation paths** when moving around obstacles.

## 4. Planned Next Steps

The next phase of the project will involve transitioning from simulation to **real-world implementation**.

Planned steps include:

### Hardware Acquisition

We aim to acquire the required hardware components including:

- Mobile robot base
- Depth camera / LiDAR sensor
- Onboard computing unit
- Motor drivers
- Battery system

### Real Sensor Integration

The simulation perception system will be replaced with real sensors such as:

- camera-based human detection
- depth sensing for obstacle detection

### Algorithm Deployment

The developed navigation and tracking algorithms will be ported to the robot platform using frameworks such as:

- Python
- ROS (Robot Operating System)

## Real World Testing

Testing will be conducted in indoor environments to evaluate:

- human following accuracy
- obstacle avoidance reliability
- navigation stability

## 5. Conclusion

The simulation successfully demonstrates the core capabilities required for a human-following robot.

The robot is able to:

- detect a human target
- follow the target smoothly
- detect obstacles
- navigate around obstacles
- operate within a realistic indoor environment

This simulation serves as a **foundation for transitioning the project toward real-world robotic implementation.**

### **Screen Recording of the current simulation**

[https://drive.google.com/file/d/1tWX2kPOAY5Rp9QA7fd-TPRvtTg-blf4p/view?usp=drive\\_link](https://drive.google.com/file/d/1tWX2kPOAY5Rp9QA7fd-TPRvtTg-blf4p/view?usp=drive_link)